

Frixoe / xor-neural-network

Public

<> Code

Issues 1

Pull requests

Actions

Projects

Security

Insights

master xor-neural-network / XOR-Net-Notebook.ipynbGo to file...

Frixoe Initial Commit

Latest commit af6eda3 on Mar 6, 2018History

1 contributor

641 lines (641 sloc) | 64.4 KB

<>RawBlame

XOR Prediction Neural Network

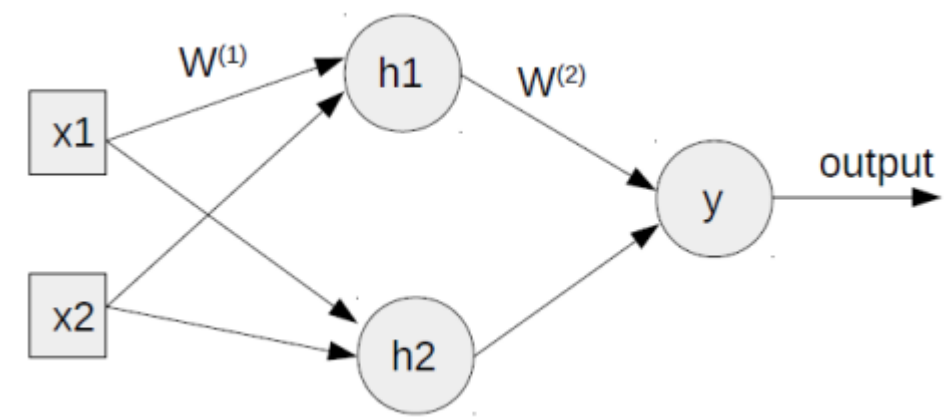
A simple neural network which will learn the XOR logic gate.

I will provide you with any links necessary so that you can read about the different aspects of this NN(Neural Network).

Neural Network Info

All information regarding the neural network:

- Input Layer Units = 2 (Can be modified)
- Hidden Layer Units = 2 (Can be modified)
- Output Layer Units = 1 (Since this is problem specific, it can't be modified)
- No. of hidden layers = 1
- Learning Algorithm = Backpropagation



Feel free to mess around with it and try out different things.

```
In [1]: import numpy as np # For matrix math
import matplotlib.pyplot as plt # For plotting
import sys # For printing
```

Neural Network Implementation

Initially, I was going to approach this in an Object Oriented manner but I think that it would be much easier to read and implement, functionally. So, let's get started.

Training Data

The XOR logic gate returns true when the number of inputs given is odd and false when they're even. Here is the simple training dataset.

```
In [2]: # The training data.
X = np.array([
    [0, 1],
    [1, 0],
    [1, 1],
    [0, 0]
])

# The Labels for the training data.
y = np.array([
    [1],
    [1],
    [0],
    [0]
])
```

In [3]:

x

```
Out[3]: array([[0, 1],
               [1, 0],
               [1, 1],
               [0, 0]])
```

In [4]:

y

```
Out[4]: array([[1],
               [1],
               [0],
               [0]])
```

Additional Parameters

These are just additional parameters which are required by the weights for their dimensions.

```
In [5]: num_i_units = 2 # Number of Input units
num_h_units = 2 # Number of Hidden units
num_o_units = 1 # Number of Output units
```

Neural Network Parameters

These are the parameters required directly by the NN. Comments should describe the variables.

```
In [6]: # The Learning rate for Gradient Descent.
learning_rate = 0.01

# The parameter to help with overfitting.
reg_param = 0

# Maximum iterations for Gradient Descent.
max_iter = 5000

# Number of training examples
m = 4
```

Weights and Biases

These are the numbers the NN needs to learn to make accurate predictions.

For the connections being made from the input layer to the hidden layer, the weights and biases are arranged in the following order: **each row contains the weights for each hidden unit**. Then, the shape of these set of weights is: *(number of hidden units X number of input units)* and the shape of the biases for this connection will be: *(number of hidden units X 1)*.

So, the overall shape of the weights and biases are:

Weights1(Connection from input to hidden layers): num_h_units X num_i_units **Biases1(Connection from input to hidden layers):** num_h_units X 1

Weights2(Connection from hidden to output layers): num_o_units X num_h_units **Biases2(Connection from hidden to output layers):** num_o_units X 1

Generating the Weights

The weights here are going to be generated using a **Normal Distribution(Gaussian Distribution)**. They will also be seeded so that the outcome always comes out the same.

```
In [7]: np.random.seed(1)
W1 = np.random.normal(0, 1, (num_h_units, num_i_units)) # 2x2
W2 = np.random.normal(0, 1, (num_o_units, num_h_units)) # 1x2

B1 = np.random.random((num_h_units, 1)) # 2x1
B2 = np.random.random((num_o_units, 1)) # 1x1
```

In [8]:

W1

```
Out[8]: array([[ 1.62434536, -0.61175641],
               [-0.52817175, -1.07296862]])
```

In [9]:

W2

```
Out[9]: array([[ 0.86540763, -2.3015387 ]])
```

In [10]:

B1

```
Out[10]: array([[0.41919451],
                [0.6852195 ]])
```

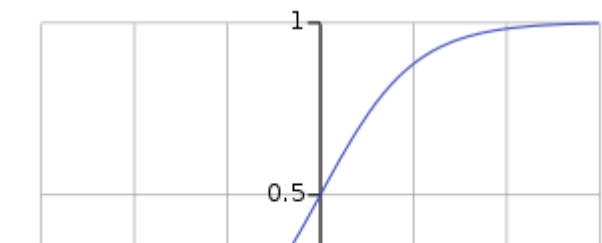
In [11]:

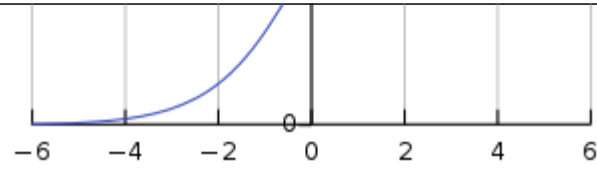
B2

```
Out[11]: array([[0.20445225]])
```

Sigmoid Function

This function maps any input to a value between 0 and 1.





In my implementation, I have added a boolean which if set to true, will return **Sigmoid Prime(the derivative of the sigmoid function)** of the input value. This will be used in backpropagation later on.

```
In [12]: def sigmoid(z, derv=False):
        if derv: return z * (1 - z)
        return 1 / (1 + np.exp(-z))
```

Forward Propagation

This is how predictions are made. Propagating the input through the NN to get the output.

In my implementation, the forward function only accepts a feature vector as row vector which is then converted to a column vector. Also, the predict boolean, if set to true, only returns the output. Otherwise, it returns a tuple of the outputs of all the layers.

```
In [13]: def forward(x, predict=False):
        a1 = x.reshape(x.shape[0], 1) # Getting the training example as a column vector.

        z2 = W1.dot(a1) + B1 # 2x2 * 2x1 + 2x1 = 2x1
        a2 = sigmoid(z2) # 2x1

        z3 = W2.dot(a2) + B2 # 1x2 * 2x1 + 1x1 = 1x1
        a3 = sigmoid(z3)

        if predict: return a3
        return (a1, a2, a3)
```

Gradients for the Weights and Biases

These variables will contain the gradients for the weights and biases which will be used by gradient descent to update the weights and biases.

Also, creating the vector which will be storing the cost values for each gradient descent iteration to help visualize the cost as the weights and biases are updated.

```
In [14]: dw1 = 0 # Gradient for W1
        dw2 = 0 # Gradient for W2

        dB1 = 0 # Gradient for B1
        dB2 = 0 # Gradient for B2

        cost = np.zeros((max_iter, 1)) # Column vector to record the cost of the NN after each Gradient Descent iteration.
```

Training

This is the training function which contains the meat of NN. This contains forward propagation and **Backpropagation**.

Backpropagation

The process of propagating the error in the output layer, backwards through the NN to calculate the error in each layer. Intuition: It's like forward propagation, but backwards.

Steps(for this NN):

1. Calculate the error in the output layer(dz2).
2. Calculate the error in the weights connecting the hidden layer to the output layer using dz2 (dW2).
3. Calculate the error in the hidden layer(dz1).
4. Calculate the error in the weights connecting the input layer to the hidden layer using dz1 (dW1).
5. The errors in the biases are just the errors in the respective layers.

Afterwards, the gradients(errors) of the weights and biases are used to update the corresponding weights and biases by multiplying them with the negative of the learning rate and scaling it by dividing it by the number of training examples.

While iterating over all the training examples, the cost is also being calculated simultaneously for each example. Then, a regularization parameter is added, although for such a small dataset, regularization is unnecessary since to perform well, the NN will have to over fit to the training data.

```
In [15]: def train(_W1, _W2, _B1, _B2): # The arguments are to bypass UnboundLocalError error
        for i in range(max_iter):
            c = 0

            dw1 = 0
            dw2 = 0

            dB1 = 0
            dB2 = 0

            for j in range(m):
                sys.stdout.write("\rIteration: {} and {}".format(i + 1, j + 1))

                # Forward Prop.
                a0 = X[j].reshape(X[j].shape[0], 1) # 2x1

                z1 = _W1.dot(a0) + _B1 # 2x2 * 2x1 + 2x1 = 2x1
                a1 = sigmoid(z1) # 2x1

                z2 = _W2.dot(a1) + _B2 # 1x2 * 2x1 + 1x1 = 1x1
                a2 = sigmoid(z2) # 1x1

                # Back prop.
                dz2 = a2 - y[j] # 1x1
                dw2 += dz2 * a1.T # 1x1 .* 1x2 = 1x2

                dz1 = np.multiply((_W2.T * dz2), sigmoid(a1, derv=True)) # (2x1 * 1x1) .* 2x1 = 2x1
                dw1 += dz1.dot(a0.T) # 2x1 * 1x2 = 2x2

                dB1 += dz1 # 2x1
                dB2 += dz2 # 1x1

            c = c + (-(y[j] * np.log(a2)) - ((1 - y[j]) * np.log(1 - a2)))
            sys.stdout.flush() # Updating the text.

            _W1 = _W1 - learning_rate * (dw1 / m) + ( (reg_param / m) * _W1)
            _W2 = _W2 - learning_rate * (dw2 / m) + ( (reg_param / m) * _W2)

            _B1 = _B1 - learning_rate * (dB1 / m)
            _B2 = _B2 - learning_rate * (dB2 / m)
            cost[i] = (c / m) + (
                (reg_param / (2 * m)) *
                (
                    np.sum(np.power(_W1, 2)) +
                    np.sum(np.power(_W2, 2))
                )
            )
        return (_W1, _W2, _B1, _B2)
```

Running

Now, let's try out the NN. Here, I have called the train() function. You can make any changes you like and then run all the kernels again. I have also plotted the cost function to visual how the NN performed.

The console printing might be off.

The weights and biases are then shown.

```
In [16]: W1, W2, B1, B2 = train(W1, W2, B1, B2)
```

Iteration: 5000 and 4

```
In [17]: W1
```

```
Out[17]: array([[ 1.32260643, -0.42332921],
                [-1.4336158 , -1.67239068]])
```

```
In [18]: W2
```

```
Out[18]: array([[ 0.25299514, -2.21317287]])
```

```
In [19]: B1
```

```
Out[19]: array([[ 0.37348207],
                [-0.22080277]])
```

```
In [20]: B2
```

```
Out[20]: array([[0.24523225]])
```

Plotting

Now, let's plot a simple plot showing the cost function with respect to the number of iterations of gradient descent.

```
In [21]: # Assigning the axes to the different elements.
        plt.plot(range(max_iter), cost)

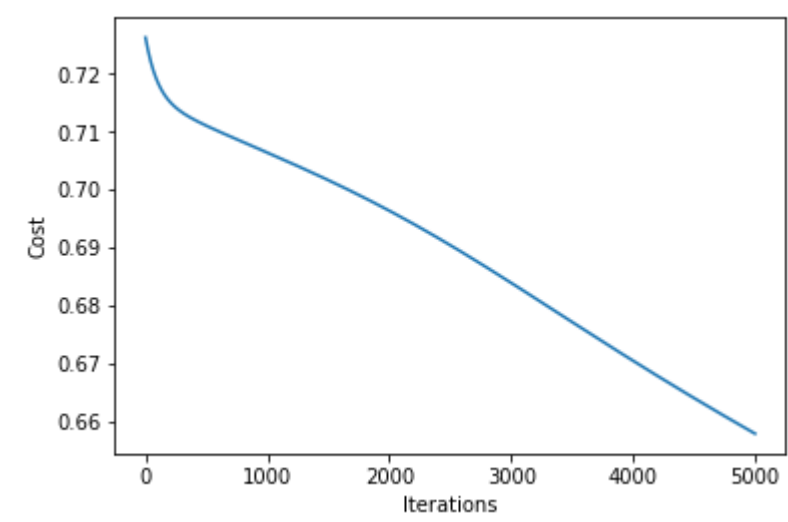
        # Labelling the x axis as the iterations axis.
        plt.xlabel("Iterations")

        # Labelling the y axis as the cost axis.
        plt.ylabel("Cost")

        # Showing the plot.
```



```
plt.show()
```



Observation

With the initial parameters, the cost function doesn't look that good. It is decreasing which is a good sign but it isn't flattening out. I have tried, multiple different values but this some seems like the best fit.

Try out your own values, run the notebook again and see what you get.